

Разработка через тестирование TDD и покрытие кода

Цель: освоить цикл Red-Green-Refactor, разрабатывать небольшие функции и классы от требований к тестам, измерять покрытие строк и ветвей, а также понимать ограничения показателя покрытия.

Этап занятия	Время	Результат
Первая пара: вводная ситуация и лекция	45 минут	Понимание TDD и покрытия
Первая пара: демонстрация цикла	25 минут	Первый завершённый цикл
Первая пара: мини-задание	20 минут	Тест до реализации
Вторая пара: практическая разработка	65 минут	Сервис и тесты
Вторая пара: покрытие, отчёт и защита	25 минут	Отчёт и вывод

Проблемная ситуация

Разработчик получил требование: «При 50 баллах студент получает зачёт». Он сразу написал функцию, затем проверил только значения 30 и 80. Код работает, но ошибка в условии `score > 50` осталась незамеченной. TDD меняет порядок работы: сначала фиксируется ожидаемое поведение в тесте, затем создаётся минимальная реализация.

1. Что такое TDD

Test-Driven Development - подход к разработке, при котором новый фрагмент поведения начинается с автоматического теста. Тест описывает маленькое требование и становится проверяемым примером ожидаемого результата.

TDD не означает, что тесты заменяют анализ требований или ручную проверку. Подход помогает разрабатывать небольшими шагами и быстро замечать, когда изменение нарушает уже реализованное поведение.

2. Цикл Red Green Refactor

Шаг	Действие	Что должно произойти
Red	Написать один тест для нового поведения	Тест падает по понятной причине
Green	Добавить минимально достаточный код	Новый и старые тесты проходят
Refactor	Улучшить структуру без изменения поведения	Все тесты продолжают проходить

Почему первый тест должен упасть

Падение подтверждает, что тест действительно проверяет ещё не реализованное поведение. Если новый тест сразу проходит, возможно, требование уже реализовано, тест проверяет не то или содержит слишком слабое ожидание.

Пример требования

Стоимость печати составляет 30 тенге за страницу. При заказе от 10 страниц включительно применяется скидка 10 процентов.

Red

```
import unittest
from print_service import print_cost

class TestPrintCost(unittest.TestCase):
    def test_discount_starts_at_ten_pages(self):
        self.assertEqual(print_cost(10), 270)
```

На первом запуске тест может завершиться ошибкой импорта или отсутствия функции. Это ожидаемый Red, если причина соответствует текущему шагу.

Green

```
def print_cost(pages):
    total = pages * 30
    if pages >= 10:
        return total * 0.9
    return total
```

Refactor

```
PRICE_PER_PAGE = 30
DISCOUNT_BOUNDARY = 10
DISCOUNT_RATE = 0.10

def print_cost(pages):
    total = pages * PRICE_PER_PAGE
    if pages >= DISCOUNT_BOUNDARY:
        total -= total * DISCOUNT_RATE
    return total
```

После рефакторинга повторно запускают весь набор тестов. Если тесты проходят, наблюдаемое поведение сохранилось.

3. Размер шага в TDD

Один цикл должен добавлять небольшое и понятное правило. Для функции стоимости печати удобно двигаться так: обычный заказ, нулевое количество, граница скидки, заказ после границы, отрицательное значение.

Очередь	Новый тест	Ожидаемый результат
1	<code>print_cost(1)</code>	30
2	<code>print_cost(0)</code>	0
3	<code>print_cost(10)</code>	270
4	<code>print_cost(11)</code>	297
5	<code>print_cost(-1)</code>	ValueError

4. TDD и тестирование после разработки

TDD	Тестирование после кода
Тест помогает уточнить интерфейс до реализации	Интерфейс уже определён кодом
Код создаётся небольшими проверяемыми шагами	Большой объём кода проверяется позже
Каждый новый шаг защищён регрессией	Непроверенные ветви могут накапливаться
Требует дисциплины и маленьких требований	Удобно для уже существующего кода

Оба подхода применяются на практике. При работе с существующим проектом сначала создают характеризующие тесты, фиксирующие текущее поведение, а затем выполняют рефакторинг.

5. Что TDD не гарантирует

- отсутствие всех дефектов;
- правильность самих требований;
- удобство пользовательского интерфейса;
- безопасность и производительность без специальных проверок;
- качество тестов, если сценарии выбраны плохо.

6. Что такое покрытие кода

Покрытие показывает, какая часть программного кода была выполнена во время тестов. Это диагностический показатель: он помогает найти участки, до которых тесты не дошли.

Вид покрытия	Что измеряется	Пример вопроса
Покрытие строк	Выполненные строки программы	Запускалась ли строка <code>raise</code> ?
Покрытие ветвей	Выполненные исходы условий	Проверялись ли <code>True</code> и <code>False</code> ?
Покрытие функций	Вызванные функции и методы	Был ли вызван <code>grant_status()</code> ?

Высокое покрытие не равно качественным тестам

```
def access_allowed(age):  
    return age >= 18  
  
def test_access():  
    access_allowed(25) # строка выполнена, но результата никто не проверил
```

Строка считается выполненной, но тест не содержит `assert`. Даже 100 процентов покрытия не доказывают правильность ожиданий и полноту тестовых данных.

7. Покрытие ветвей

```
def exam_result(score):  
    if score >= 50:  
        return "Зачёт"  
    return "Незачёт"
```

Для выполнения обеих ветвей нужны как минимум два сценария: значение ниже 50 и значение 50 или выше. Для проверки границы разумно использовать 49, 50 и 51.

8. Измерение с coverage.py

Coverage.py запускает тесты, собирает сведения о выполненных строках и создаёт текстовый или HTML-отчёт. Установка выполняется один раз в активном окружении проекта.

```
python -m pip install coverage
```

```
# Запуск unittest с измерением покрытия
coverage run -m unittest discover
```

```
# Текстовый отчёт
coverage report -m
```

```
# Измерение ветвей
coverage run --branch -m unittest discover
coverage report -m
```

```
# Подробный HTML-отчёт
coverage html
```

HTML-отчёт создаётся в папке htmlcov. В нём выполненные строки выделяются отдельно от пропущенных. При измерении ветвей отображаются также частично проверенные переходы.

Как читать текстовый отчёт

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
student_service.py	18	3	8	2	78%	14-16
test_student.py	24	0	2	0	100%	
TOTAL	42	3	10	2	87%	

- Stmts - учитываемые строки кода.
- Miss - строки, которые не выполнялись.
- Branch - возможные переходы условий.
- BrPart - частично пройденные ветвления.
- Missing - номера строк, требующие анализа.

9. Правильная реакция на непокрытую строку

1. Понять назначение строки или ветви.
2. Проверить, существует ли реальное требование для этого поведения.
3. Добавить содержательный тест с ожидаемым результатом.
4. Если код недостижим или не нужен, удалить его обоснованно.
5. Запустить весь набор и повторно измерить покрытие.

10. TDD для класса

Рассмотрим класс учёта посещаемости. Сначала тестируем создание объекта, затем добавление посещения, запрет отрицательных значений и вычисление процента.

```
class TestAttendance(unittest.TestCase):
    def test_new_student_has_zero_lessons(self):
        student = Attendance("Алия")
        self.assertEqual(student.total_lessons, 0)
        self.assertEqual(student.attended_lessons, 0)

    def test_register_attendance(self):
        student = Attendance("Алия")
        student.register_lesson(attended=True)
        self.assertEqual(student.total_lessons, 1)
        self.assertEqual(student.attended_lessons, 1)
```

После первого Red создают конструктор с двумя счётчиками. После второго Red добавляют `register_lesson()`. Каждый следующий метод появляется из нового требования, выраженного тестом.

11. Когда завершать цикл

- новый тест проходит;
- все ранее написанные тесты также проходят;
- код соответствует текущему требованию;
- имена и структура понятны;
- нет очевидного дублирования;
- следующее правило можно описать отдельным тестом.

Мини-задание первой пары

Требование: функция `delivery_cost(distance)` возвращает 500 тенге при расстоянии до 5 км включительно и 100 тенге за каждый дополнительный километр. Отрицательное расстояние вызывает `ValueError`.

1. Напишите первый падающий тест для 5 км.
2. Создайте минимальную реализацию.
3. Добавьте тест для 6 км и расширьте код.
4. Добавьте тест отрицательного значения.
5. Выполните рефакторинг имён и запустите все тесты.

Сервис регистрации на курсы по циклу TDD

Ситуация: колледжу ЕКЕВ требуется небольшой сервис регистрации студента на дополнительный курс. Реализацию нужно создавать постепенно, начиная с тестов.

Организация файлов

```
course_service.py      # реализация
test_course_service.py # тесты unittest
```

Правила работы

- Каждый этап начинается с нового теста.
- Перед изменением кода убедитесь, что новый тест падает по ожидаемой причине.
- Добавляйте минимальную реализацию.
- После Green запускайте все предыдущие тесты.
- Рефакторинг выполняйте только при зелёном наборе.
- После каждого этапа фиксируйте Red, Green и изменение кода.

Этап 1. Создание курса

Класс Course принимает название и количество мест. При создании enrolled равно нулю. Название не может быть пустым, количество мест должно быть положительным целым числом.

- Тест корректного названия и capacity.
- Тест начального enrolled == 0.
- Тест ValueError для пустого названия.
- Тест ValueError для capacity 0 и отрицательного значения.

Этап 2. Свободные места

Метод available_places() возвращает capacity - enrolled. Сначала напишите тест нового курса, затем реализуйте метод.

Этап 3. Регистрация студента

Метод `enroll()` увеличивает `enrolled` на 1 и возвращает оставшееся количество мест. Если мест нет, метод вызывает `ValueError` с понятным сообщением.

- Регистрация одного студента.
- Последовательная регистрация нескольких студентов.
- Регистрация на последнее свободное место.
- Попытка регистрации сверх `capacity`.

Этап 4. Отмена регистрации

Метод `cancel_enrollment()` уменьшает `enrolled` на 1. Если никто не зарегистрирован, вызывается `ValueError`.

- Отмена после одной регистрации.
- Повторная отмена при `enrolled == 0`.
- Проверка восстановления свободного места.

Этап 5. Заполненность курса

Свойство `is_full` возвращает `True`, когда `enrolled` равно `capacity`, иначе `False`. Проверьте пустой, частично заполненный и полностью заполненный курс.

Этап 6. Дочерний класс `IntensiveCourse`

`IntensiveCourse` наследует `Course` и дополнительно хранит `hours_per_week`. Значение должно быть от 6 до 20 включительно. Метод `workload_level()` возвращает «средняя» при 6-10 часах и «высокая» при 11-20.

- Проверка наследования через `assertIsInstance`.
- Границы 6, 10, 11 и 20.
- Ошибочные значения 5 и 21.
- Наследованные `enroll()`, `available_places()` и `is_full`.

Этап 7. Измерение покрытия

```
coverage run --branch -m unittest discover
coverage report -m
coverage html
```

1. Запишите общий процент покрытия строк.
2. Найдите пропущенные строки и частичные ветви.
3. Для каждого пропуска решите: нужен тест или код лишний.
4. Добавьте содержательные тесты.
5. Повторите измерение и сравните результаты.

Журнал TDD

Этап	Red: какой тест упал	Green: что добавлено	Refactor: что улучшено
1. Course			
2. available_places			
3. enroll			
4. cancel_enrollment			
5. is_full			
6. IntensiveCourse			

Отчёт покрытия

Показатель	До дополнительных тестов	После анализа
Покрытие строк		
Покрытие ветвей		
Пропущенные строки		
Частичные ветви		
Количество тестов		

Описание одного найденного пробела

Поле	Ответ
Непокрытая строка или ветвь	
Какое требование связано с ней	
Какой тест добавлен	
Почему тест содержательный	
Результат повторного запуска	

Что нужно сдать

- `course_service.py` с реализацией.
- `test_course_service.py` с тестами.
- Заполненный журнал TDD.
- Текстовый отчёт `coverage report -m`.
- Снимок HTML-отчёта по одной выбранной функции или методу.
- Вывод из 4-5 предложений: что показало покрытие и какие тесты были добавлены.

Критерии оценивания

Критерий	Баллы
Соблюдён цикл Red-Green-Refactor	2
Проверены обычные, граничные и ошибочные случаи	2
Корректно реализованы <code>Course</code> и <code>IntensiveCourse</code>	2
Измерены строки и ветви, отчёт интерпретирован	2
Полный набор тестов проходит, материалы оформлены	2

Дополнительное задание

Добавьте `waitlist`. Когда курс заполнен, `enroll(add_to_waitlist=True)` не создаёт ошибку, а увеличивает очередь. После `cancel_enrollment()` первый студент из очереди автоматически занимает освободившееся место. Сначала зафиксируйте каждое правило отдельным тестом.

Вопросы для защиты

1. Почему новый тест в TDD сначала должен упасть?
2. Чем Green отличается от окончательного качественного решения?
3. Почему рефакторинг выполняется при зелёном наборе?
4. Что означает покрытие строк?
5. Почему 100 процентов не доказывают отсутствие ошибок?
6. Какие тесты нужны для обеих ветвей `is_full`?
7. Как покрытие помогает найти недостающий сценарий?

Источники для самостоятельной работы

Документация Python: `unittest` — docs.python.org/3/library/unittest.html

Документация `Coverage.py` — coverage.readthedocs.io